私域麻将馆运营 Agent：完整系统设计与场景实现

这不是一份概念性说明，也不是 Demo 功能清单。本文以当前 `mahjong_agent_runtime` 代码为唯一事实来源，解释消息如何进入系统、模型如何围绕目标规划工具、后端如何保证状态与数据安全，以及真实麻将馆运营中每类问题如何被建模、执行、审计和回归验证。

Goal-driven Agent | ReAct-style Loop | Tool Calling | Context Engineering | Human-in-the-Loop | WeChaty 灰度通道 | SQLite + 可选 Redis | Trace / Badcase / Eval

> **项目根目录：** `<repository-root>`
> **当前主运行时：** `src/mahjong_agent_runtime` 。旧 Demo 或历史架构不再作为系统事实。

1. 项目定位与目标

1.1 解决什么问题

麻将馆老板日常要在大量私聊和群聊里完成意图识别、条件确认、现有局匹配、候选人邀约、时间协商、人数维护、局状态推进和客户关系维护。这个过程不是单次问答，而是跨用户、跨时间、多轮、带状态和副作用的持续任务。

系统目标不是“能聊麻将”，而是**接受客户输入后，自主判断下一步需要什么信息和工具，持续推进直到当前目标完成、等待用户、需要人工或明确失败。**

95%
目标自动回复率
常规咨询、组局、候选确认、状态查询等消息无需人工编写。

数百人
单店用户规模
优先支持一台 MacBook 本地部署和单店私域运营。

可追溯
每次决策有证据
trace、上下文、模型动作、工具结果、状态变更和审查结果均可回放。

1.2 当前已经形成的闭环

接收消息 → 聚合碎片 → 构建上下文 → 模型规划 → 调用工具 → 结果回喂 → 动态改计划 → 生成回复 → 内容审查 → 外发或待审批

局和客户不是 prompt 中的临时文本，而是持久化领域对象；工具不是模型直接执行的任意函数，而是经过 schema、权限、状态、幂等和风险校验的后端能力。

2. 设计原则与边界

原则一：模型负责理解与提出动作，后端负责事实与副作用。
模型可以判断目标、生成计划、选择工具和写话术；不能直接写数据库、伪造工具结果或绕过状态机。

原则二：不以 badcase 为理由无限增加业务 if-else。
业务语义交给模型、领域数据和工具结果表达；后端硬规则只保护合法性、权限、幂等、一致性、风险和审计。

原则三：局状态、人数、房态、邀约状态必须来自事实源。
客户可见回复不能凭模型记忆猜测。任何“现在缺几人”“有没有房”“邀请了谁”都应基于工具结果或持久化状态。

原则四：可恢复比单次聪明更重要。
模型超时、非法 JSON、工具参数错误、节点重启、重复消息和并发抢位都要有明确恢复路径。

2.1 确定性代码与模型能力的边界

交给模型	交给后端
理解自然语言、错别字、隐含意图和上下文指代	消息幂等、同会话顺序、并发锁和版本控制
根据当前目标生成或调整行动计划	工具 schema、权限、风险级别和参数校验
决定调用哪个工具以及调用顺序	数据库事务、唯一约束、状态机和原子更新
根据工具结果判断下一步	房态、局状态、参与人数等权威事实计算
生成自然、简洁、符合老板语气的文本	客户可见内容审查、外发开关和人工审批
提炼摘要和候选长期记忆	摘要预算触发、证据保存、置信度阈值与落库

2.2 为什么它不是传统固定 Workflow

固定 Workflow 会预先写死“先查房态、再问档位、再查局、再找候选人”。当前系统只定义目标、工具和安全边界，模型可以根据上下文跳过不必要步骤。例如老客户已经有稳定画像时，不必重复追问档位；明确咨询现有局时，也不应先创建新局。

同时它也不是“完全无约束的自主模型”。生产 Agent 必须在确定性执行边界内运行，所以更准确的描述是：**目标驱动 Agent + 受控工具执行环境**。

3. 当前系统架构

```
微信 / Web 控制台 / 未来其他渠道
                │
                ▼
    Channel Adapter + Message Envelope
                │
                ▼
  Input Aggregator（conversation + sender 维度）
                │
                ▼
  Input Gate（业务 / 闲聊 / 等待补充 / 忽略）
                │
                ▼
  Runtime Boundary
  ├─ traceId / message idempotency
  ├─ conversation lock / run version
  ├─ supersede 旧待发结果
  └─ post-turn summary
                │
                ▼
  Agent Loop
  ├─ ContextBuilder
  ├─ DeepSeek LLM
  ├─ Action Contract Parser
  ├─ ToolGateway
  ├─ Tool Results → 回喂模型
  ├─ Progress Monitor
  └─ Copywriter / Content Reviewer
                │
                ▼
  Domain Store
  ├─ SQLite：事实源、状态、审计、幂等
  └─ Redis（可选）：分布式锁、短期缓存、热点状态
                │
                ▼
  Outbox / WeChaty 外发 / 人工审批
```

3.1 模块职责

模块	职责	关键代码
HTTP 与渠道适配	接收控制台和 WeChaty 消息，标准化消息信封，管理外发开关	`scripts/agent_runtime_app.py`
Runtime 边界	加锁、幂等、trace、版本、旧输出作废、调用主循环	`src/mahjong_agent_runtime/runtime.py`
主循环	构建上下文、调用模型、执行工具、回喂结果、停止判断	`src/mahjong_agent_runtime/loop.py`
生命周期处理	上下文预压缩、会话后摘要、运行生命周期 hook	`src/mahjong_agent_runtime/lifecycle.py`
动作处理	区分工具动作与终态回复，执行审查与状态收集	`src/mahjong_agent_runtime/processing.py`
上下文	按预算装配近期对话、checkpoint、画像、局、任务记忆和工具	`src/mahjong_agent_runtime/context.py`
工具网关	注册工具、对模型暴露 schema、校验并执行副作用	`src/mahjong_agent_runtime/tools.py`
持久化	SQLite 表、事务、原子抢位、幂等账本和查询	`src/mahjong_agent_runtime/sqlite_store.py`
摘要与记忆	生成 checkpoint，写任务记忆和候选长期画像	`summary.py`、`memory.py`
进展检测	识别重复观察、短周期循环和连续无进展	`src/mahjong_agent_runtime/progress.py`

3.2 为什么没有直接迁到 LangGraph / DeepAgents

框架可以提供通用 loop、checkpoint、graph 和人工节点，但不会自动解决麻将领域的状态、人数席位、房态事务、跨工具参数一致性、隐私边界和真实话术。当前项目最难的部分恰好是这些领域语义和生产约束。

因此当前选择先把领域内核、工具契约和评测做实；当出现跨任务持久编排、复杂人工队列、可视化图执行或大量可复用子图时，再以 PoC 方式评估框架迁移，而不是为了使用框架重写已验证的业务内核。

4. 一条消息的完整链路

1. 渠道适配器接收消息，生成或保留 `source_message_id`、`conversation_id`、`sender_id`、引用消息和渠道元数据。
2. 消息按 `conversation_id + sender_id` 进入碎片聚合器，避免群里不同人的消息串在一起。

3. 输入门禁判断当前聚合文本属于业务、闲聊、应该继续等待补充还是忽略。
4. 正式进入 Runtime 后生成 `trace_id`，检查消息幂等，并获取会话协调锁。
5. 会话版本递增；同一会话此前尚未外发的旧回复、邀约草稿被标记为 `superseded`。
6. `ContextBuilder` 从 SQLite 读取近期 turn、checkpoint、画像、关系、任务记忆、活跃局、待发草稿和可用工具。
7. 若估算上下文接近主模型预算的 85%，先生成 checkpoint，再重建上下文。
8. 模型输出结构化 Action：目标、当前状态、计划、工具调用或客户回复。
9. 若有工具调用，ToolGateway 校验并执行；工具结果作为 `previous_tool_results` 回喂下一轮模型。
10. 进展监控器检查重复工具结果、短周期循环和连续无进展；必要时要求重规划或中止。
11. 若模型准备结束，话术模型优化表达，审查模型核对隐私、事实和内部信息泄露。
12. 最终文本根据通道策略自动外发、进入待审批 outbox，或在高风险情况下转人工。
13. 输入、上下文审计、模型动作、工具结果、状态变更、审查和最终输出全部写入同一 trace。

4.1 一次多工具任务的时序

```
User: “五点0.5，173，无烟”
  │
  ├─ Gate: business，信息可行动
  │
  ├─ Agent: search_current_games(requirement)
  │      └─ Tool: 返回无匹配局 + 实际查询 requirement
  │
  ├─ Agent: check_room_availability(17:00, end_time)
  │      └─ Tool: 返回有房
  │
  ├─ Agent: create_game(requirement, party_size=1)
  │      └─ Tool: 原子落库，发起客户计入席位
  │
  ├─ Agent: search_customers(requirement, exclusions)
  │      └─ Tool: 返回确定性排序和关系约束
  │
  ├─ Agent: create_invite_drafts(game_id, candidates)
  │      └─ Tool: 仅创建草稿，不直接发送
  │
  ├─ Agent: reply_to_user = “好，我帮你问问。”
  │
  ├─ Copywriter + Reviewer
  │
  └─ Outbox / 自动外发开关
```

5. 模型角色划分

系统会多次调用模型，但不是多 Agent 系统。只有主 Agent 持有业务目标并能连续选择工具；其余调用都是边界明确、一次性完成的模型任务。

角色	输入	输出	是否持有业务目标
主 Agent	完整业务上下文、工具列表、工具结果、计划与状态	Action Contract，可能包含工具调用	是
输入门禁	聚合消息、近期必要上下文、是否已经静默	business / casual / wait / ignore	否
话术生成器	主 Agent 原始回复、权威工具结果、语气要求	不改变事实的自然文本	否
内容审查器	所有客户可见文本、允许公开的事实和原文	通过/拒绝、违规项、安全改写建议	否
摘要器	旧 checkpoint、近期 turn、活跃业务对象	结构化 checkpoint	否
长期记忆提炼器	会话证据、现有画像	候选事实、证据、置信度	否

> **不保存隐藏推理链。**日志只保存模型输出的 `reasoning_summary`、行动计划和可审计证据，不要求或持久化模型内部逐 token 思维过程。

6. 消息接入与碎片聚合

6.1 统一消息信封

无论消息来自 Web 控制台、WeChaty、群聊还是私聊，进入主流程前都转换为统一的 `UserMessage`。渠道层保留原始 payload 供审计，但主 Agent 只依赖稳定字段。

```
{
  "conversation_id": "wechaty:contact:@xxx",
  "sender_id": "@xxx",
  "sender_name": "公开昵称或空",
  "text": "老板，今天下午有人打麻将吗",
  "message_id": "渠道原始消息 ID",
  "channel": "wechaty",
  "is_room": false,
  "quoted_message": null,
  "received_at": "2026-07-19T14:00:00+08:00"
}
```

代码中出现的 `runtime_trial`、`zhang`、`张哥`只是控制台未提供字段时的测试默认值，不是生产用户绑定。真实渠道必须传入真实会话和发送者标识。

6.2 为什么要按 conversation + sender 聚合

只按 `conversation_id` 聚合会把群内多个用户连续发送的消息拼在一起；只按 `sender_id` 又会混淆同一用户在不同群和私聊中的语境。因此 batch key 使用：

```
batch_key = conversation_id + ":" + sender_id
```

每条新碎片会被持久化，聚合批次版本递增，静默截止时间重新计算。聚合后的输入保留所有 `source_message_id`，以便追踪一条 Agent 任务由哪些原始消息组成。

6.3 30 秒静默窗口不是无脑延迟

输入门禁先判断当前文本是否已经可以行动。明确的“下午两点 0.5 无烟杭麻，173”无需等待；“老板”“今天下午”这类碎片可以等待用户继续输入。门禁会收到 `has_already_waited`，防止无限静默。

调度器在名义 30 秒静默期之外保留默认 5 秒边界宽限。该宽限不属于业务语义，只用于避免用户恰好每 30 秒补充一条时，静默任务与新消息同时抢占批次。入口模型仍然只看到名义静默期，日志同时记录名义值、宽限值和实际调度值。

门禁结果	含义	处理
`business`	已经能识别业务目标或必须追问	立即进入主 Agent
`casual`	闲聊，与当前业务目标无直接推进关系	走轻量闲聊生成与内容审查
`wait`	像碎片开头，用户大概率还会补充	持久化并设置 30 秒延时检查
`ignore`	空消息、自发回声、不支持的系统事件	记录原因，不进入主流程

延时任务具有持久化 due time 和 processing lease。进程重启后可以重新扫描到期批次；同一批次只能被一个 worker 领取。若已经静默过，门禁不能再次返回无期限等待，而要进入正式判断、追问或忽略。

6.4 新消息如何打断旧回复

用户可能在 Agent 生成回复但尚未外发时补充“0.5 或 1 都行、烟也都可”。Runtime 为会话维护递增版本。新消息进入后：

1. 创建新的 `run_id + run_version`。
2. 旧版本尚未发送的回复、邀约草稿和 outbox 记录标记为 `superseded`。
3. 旧运行即使稍后完成，也会在落库或外发前因版本过期被拒绝。
4. 新运行使用补充后的完整聚合消息重建上下文。

> 这相当于“撤回并重新编辑”，但实现方式不是操作微信已经发送的消息，而是在系统外发前保证只有最新版本有发送资格。

7. 主 Agent Loop

7.1 主循环的机制职责边界

主循环采用稳定的机制内核：`build_context → call_llm → execute_tools → append_tool_results`。这不是追求代码行数少，而是落实机制与策略分离。上下文生命周期、并发协调、客户可见内容审查、持久化和领域副作用由可独立测试的边界模块承担；新增业务能力不得通过向 Runtime 或主循环追加场景分支实现。

```
for step in range(max_steps):
    context = context_builder.build(message, previous_tool_results)

    if context_near_budget(context):
        summarize_and_checkpoint()
        context = context_builder.build(message, previous_tool_results)

    action = llm.call(context)
    action = contract_parser.validate(action)

    if action.tool_calls:
        results = tool_processor.execute(action.tool_calls)
        previous_tool_results = results
        progress_monitor.observe(action, results)
        continue

    final_reply = reply_processor.process(action.reply_to_user)
    return final_reply

return safe_stop("达到最大步骤，转人工或保守结束")
```

7.2 Runtime 和 Loop 为什么分开

Runtime Boundary	Agent Loop
每条请求只做一次的外围责任	围绕目标反复执行的推理与工具循环
消息幂等、会话锁、trace、版本	上下文、模型、动作、工具结果、停止条件
新输入作废旧运行结果	根据工具结果动态修改计划
会话结束后摘要和结果持久化	无进展检测和最大步数

7.3 状态转换如何收集

```
transitions = [
    transition
    for result in tool_results
    if not result.deduplicated
    for transition in result.state_transitions
]
```

只有真正执行的工具结果产生新的状态转换。若本次调用命中幂等账本，`deduplicated=true`，其历史结果可以返回给模型理解，但不能再次写一份状态审计，否则同一动作会被错误记录两次。

7.4 何时停止

停止不是看模型有没有写一段话，而是看输出契约中的目标状态和停止原因：

- `needs_tool`：不能停止，必须至少调用一个工具。
- `waiting_user`：缺少必须由用户提供的信息，可以回复追问并停止本轮。
- `completed`：本轮目标已完成或已给出确定答复。
- `needs_human`：高风险、冲突或连续失败，需要人工。
- `unknown`：无法可靠判断，但必须给出安全的客户可见回应。

8. 上下文构建

8.1 ContextBuilder 在什么时候读取

每次主 Agent step 调模型前都会重新构建上下文，而不是在请求开始时只读一次。这样工具刚更新的局状态、候选确认和房态能在下一步立即可见。工具结果还会以 `previous_tool_results` 单独回喂，避免模型只依赖数据库快照猜测刚刚执行了什么。

8.2 当前 payload 包含什么

字段	内容	解决的问题
`current_message`	本轮标准化消息	明确当前用户真正说了什么
`recent_conversation`	按预算保留的 user/assistant/tool turns	理解“可以”“六点”等短回复
`conversation_checkpoint`	旧对话摘要、事实和待办	长对话压缩后不丢目标
`recovered_task_contexts`	同会话引用或可信预约唤醒所指向任务的 checkpoint/受限原文	恢复旧目标但不污染当前任务
`conversation_state`	run/version、会话运行状态	识别旧输出和当前任务阶段
`quoted_message_context`	已解析的引用消息及业务绑定	知道“可以”是在回复哪条邀请
`message_reference_contract`	引用绑定优先级和未解析状态	防止把闲聊短句错绑到活跃局
`sender_profile`	长期画像和稳定偏好	老客户少追问
`sender_relationships`	打过、回避、关系证据	候选过滤与隐私回答
`task_memories`	当前任务内有效的事实	“本局不和 C 打”立即生效
`pending_memory_candidates`	尚未晋升的长期画像候选	避免一次表达污染画像
`active_games`	当前会话关联的完整局对象	状态推进和计划
`active_game_visible_summaries`	只含可公开事实的局摘要	话术和审查不泄露内部数据
`sender_active_game_memberships`	发送者已加入哪些局、占几席	防止重复加入和人数重复计数
`active_parties`	联系人和所代表席位	一个联系人代表两个人
`outbound_message_drafts`	有效待发草稿	避免重复邀请和状态不一致

`available_tools`	本轮允许使用的工具 schema	动态限定模型能力
`previous_tool_results`	上一 step 的权威工具结果	根据观察继续 ReAct 循环
`planning_contract`	计划和目标状态规范	让计划可审计、可调整
`output_contract`	主模型 JSON 契约	后端可校验和执行

8.3 近期对话如何裁剪

系统先按 `(conversation_id, task_context_id)` 筛选当前业务任务，再最多考察该任务最近 60 个 turn，并按从新到旧的顺序装入，直到估算达到约 4000 token 的近期对话预算。这样繁忙群聊不会把未来预约原文挤出任务窗口。已被 checkpoint 覆盖的旧 turn 不再重复加入；当前 trace 已通过 `previous_tool_results` 提供的工具消息也不会重复塞入。

```
selected = []
used_tokens = 0
for turn in reverse(recent_turns):
    cost = estimate_tokens(turn)
    if used_tokens + cost > recent_conversation_budget:
        break
    selected.append(turn)
    used_tokens += cost
return reverse(selected)
```

8.4 token 估算从哪里来

`estimated_tokens` 是工程估算，不是供应商最终账单。实现会根据中文字符、英文片段、JSON 结构和固定消息开销估算；预算控制宁愿保守，也不应等 API 返回 context length error 才处理。真正模型响应中的 usage 若可用，会另外记录用于成本核算和校准估算器。

> **关键区别：**检索决定“取哪些事实”，裁剪决定“保留哪些原文”，摘要决定“旧信息如何压缩”。三者不是同一件事。

8.5 未来预约与历史引用如何恢复

四小时只轮换普通空闲任务，不删除历史。未来预约的定时任务持久化来源 `task_context_id`；用户引用同一会话的旧消息时，系统用 `source_message_id` 反查所属任务。恢复时优先读取任务 checkpoint，没有摘要才加载受预算限制的原始 turns。恢复证据只帮助理解目标，人数、房态、局状态和邀约结果仍需查数据库或调用工具。跨 conversation 引用不会导入原文或摘要。

9. 摘要与 Checkpoint

9.1 什么时候生成摘要

有两条触发路径：

1. **会话后常规触发：**总 turn 至少 12、距上次摘要至少 6 个新 turn、近期原文估算达到 3000 token。
2. **主模型调用前紧急触发：**完整上下文估算达到主模型单次预算的 85%，不再等待 turn 数条件，先摘要并重建上下文。

主模型默认单次上下文预算为 24000 estimated tokens。85% 阈值为系统留出了 system prompt、工具 schema、输出和估算误差的安全空间。

上述 turn 数、距上次摘要的轮数和 token 估算都先按 `task_context_id` 选定任务，再进行统计。普通消息使用本轮任务；可信的未来预约唤醒使用调度任务保存的来源任务 ID。这样同一个微信会话中上午、下午和未来预约的摘要不会互相污染。

9.2 摘要输入与输出

摘要输入	说明
existing checkpoint	在旧摘要基础上增量更新，而非每次从零总结
bounded recent conversation	最多考虑 80 turn，摘要输入最多约 6000 token
active games / invites / outbound drafts	用权威对象校验摘要不能把已取消局写成活跃
summary contract	要求 summary、facts、open_questions、confidence 和证据来源

```
{
  "summary": "用户正在询问今晚无烟杭麻；已查到一个邻近局，等待用户确认。",
  "facts": [
    {"key": "preferred_stakes", "value": ["0.5", "1"], "source_turn_ids": [31]}
  ],
  "open_questions": ["用户是否接受 19:00 开始"],
  "confidence": 0.91
}
```

9.3 Checkpoint 存什么、不存什么

Checkpoint 保存 `conversation_id`、`task_context_id`、摘要文本、结构化事实、未解决问题、来源 trace、更新时间和置信度。当前会话投影写入 `runtime_conversation_checkpoints`，历史任务归档写入 `runtime_task_context_checkpoints`；升级旧 SQLite 时会根据 checkpoint 内的任务 ID 幂等回填历史任务表。它是模型上下文的压缩产物，不是业务事实源。

- 局状态、人数、房态必须继续查询数据库，不能以摘要为准。

- 摘要中出现不存在或已归档的 game id 会被校验拒绝。
- 置信度低于 0.6 的摘要不覆盖已有 checkpoint。
- 最多保留 10 个开放问题，摘要文本默认限制约 800 字。
- 用户可通过控制台清空会话状态与 checkpoint，便于重新测试。

9.4 如何发现摘要损坏

摘要不是一次生成后永久可信。后续工具查询结果优先于 checkpoint；若摘要说“还缺 2 人”而工具返回“已满”，模型必须使用工具事实。评测还会覆盖摘要前后目标、关键约束、开放问题和业务对象引用是否一致。

10. LLM Contract

10.1 Contract 的通俗解释

Contract 就是模型和后端之间的接口协议。Prompt 告诉模型“应该怎样思考和做事”，Contract 则告诉它“必须用什么字段交付结果”。没有 Contract，后端只能从自然语言中猜“模型是不是想调用工具”。

10.2 主模型输出

```
{
  "goal": "帮用户找到或组建合适的麻将局",
  "objective_status": "needs_tool",
  "reasoning_summary": "先查询符合档位和烟况的现有局。",
  "objective_state": {
    "known": ["玩法=杭麻", "档位=0.5", "烟况=无烟"],
    "unknown": ["准确开始时间"],
    "constraints": ["用户接受人齐开"]
  },
  "objective_plan": [
    {"step": "查询现有局", "status": "in_progress"},
    {"step": "无匹配时建立新局", "status": "pending"}
  ],
  "plan_revision_reason": "",
  "reply_to_user": "",
  "tool_calls": [{
    "name": "search_current_games",
    "arguments": {"requirement": {}},
    "reason": "需要读取当前真实局池"
  }],
  "needs_human": false,
  "stop_reason": {
    "can_stop": false,
    "why": "尚未获得局池结果",
    "pending_work": ["执行 search_current_games"],
    "depends_on_tool_results": true
  }
}
```

10.3 必须满足的不变量

- `needs_tool` 必须有至少一个 `tool_call`，且 `reply_to_user` 必须为空。
- 终态不能同时包含工具调用，必须有非空客户回复。
- `needs_human=true` 必须与 `objective_status=needs_human` 一致。
- 不能停止时 `pending_work` 必须非空；终态 `can_stop` 必须为 true。
- 工具调用的 `arguments` 还会被 ToolGateway 二次 schema 校验。
- 模型给出的幂等键不具权威性，真正 key 由后端根据 trace、工具和参数派生。

10.4 非法 Contract 如何处理

1. 先做 JSON 提取和结构校验，不执行任何副作用。
2. 可修复的纯格式错误回喂模型，要求按契约重写。
3. 只有一个领域无关的窄修复：字段均表示终态、回复非空、无工具，但状态误填 `needs_tool` 时，可规范为 `completed` 并记审计日志。

4. 不能通过“修复 JSON”猜测业务人数、时间或烟况；业务不确定必须再问模型或用户。

11. 工具系统

11.1 工具从定义到执行

1. 后端用名称、描述、JSON schema、风险级别和执行函数注册工具。
2. ContextBuilder 根据当前运行策略把允许的工具列表放入本轮上下文。
3. 模型只提出 `tool_call`，不直接运行 Python 或 SQL。
4. ToolGateway 校验工具是否存在、参数类型、业务权限、状态合法性和幂等。
5. 执行函数在事务中读写领域对象，返回结构化结果和状态转换。
6. 结果进入 trace 并回喂主模型；模型再决定下一步。

11.2 当前 13 个工具

工具	类型/风险	作用与关键约束
`check_room_availability`	只读 / 低	查询开始与结束区间的房态；时间必须明确。
`reserve_room`	写状态 / 中	原子占用房间；创建预订前必须先查房态。
`search_current_games`	只读 / 低	按结构化需求查询可行动局，并返回加入前后缺口投影。
`search_customers`	只读 / 低	按玩法、档位、时间、烟况、疲劳和关系做确定性排序。
`create_game`	写状态 / 中	只创建局记录；发起方占用实际席位，不等于已订房或已发消息。
`create_invite_drafts`	写草稿 / 中	为候选人创建邀约草稿；不会直接发送。
`create_outbound_message_drafts`	写草稿 / 中	创建通用多通道待发消息。
`record_candidate_reply`	写状态 / 中	记录确认、拒绝、协商、未回复、到店及代表席位。
`update_game_requirement`	写状态 / 中	更新尚未成局且已经协商确认的时间、时长、玩法、档位或烟况；不能修改参与者和座位。
`update_game_status`	写状态 / 中	只允许合法状态迁移，不能借此修改组局需求。
`record_badcase`	写审计 / 低	把线上或测试问题结构化归档。
`record_user_memory`	写状态 / 中	任务记忆立即生效；长期画像先进入候选区。
`update_context_checkpoint`	写状态 / 中	写入结构化会话 checkpoint。

11.3 为什么不提供通用 SQL 工具

通用 SQL 会让模型绕过权限、行级隔离、状态机、幂等和字段脱敏，也难以为每个副作用定义审计语义。系统只暴露业务能力工具；查询和更新由工具内部的结构化存储 API 完成。

11.4 工具参数不合法时怎样让模型知道

ToolGateway 不执行工具，而是生成一个普通 `ToolResult`：

```
{
  "name": "create_game",
  "allowed": false,
  "called": false,
  "error": {
    "type": "schema_validation_error",
    "field": "requirement.known_player_count",
    "message": "must be an integer between 1 and 4"
  }
}
```

该结果作为 `previous_tool_results` 进入下一轮 ContextBuilder。模型看到错误后修正参数、改用其他工具或追问用户。后端不会静默吞掉错误，也不会自行猜参数。

11.5 跨工具参数一致性

如果模型先按 `16:00 / 0.5 / 173 / 无烟` 查局，下一步创建时改成"人齐开 / 缺三人"，Tool Consistency 会比较上一工具实际执行的 requirement 与新写操作的关键字段。冲突时拒绝落库并回喂差异：

```
{
  "error": "tool_requirement_inconsistent",
  "differences": {
    "start_mode": {"searched": "fixed", "create": "asap_when_full"},
    "planned_start": {"searched": "16:00", "create": null}
  }
}
```

这是通用的"读后写一致性"保护，不是针对一句话写的 if-else。

11.6 无依赖只读工具如何并行

模型在同一步中为工具调用提供唯一 `call_id` 和 `depends_on`。后端校验依赖图后按波次执行；只有注册为 `read_only + parallel_safe` 的工具才能在同波线程池中并行。当前允许并行的是房态查询、现有局查询和候选人查询。建局、修改参与者、草稿与审计写入始终串行。

```
[
  {"call_id":"rooms", "depends_on":[], "name":"check_room_availability"},
  {"call_id":"games", "depends_on":[], "name":"search_current_games"},
  {"call_id":"create", "depends_on":["rooms","games"], "name":"create_game"}
]
```

前置失败时，后续依赖不执行，失败证据会回喂模型重新规划。并行完成顺序不影响上下文，结果会按模型原始调用顺序重排。依赖图中每个调用都必须有唯一 `call_id`；省略或返回 `null` 的 `depends_on` 会归一化为空依赖，而未知依赖、自依赖和环仍会被拒绝。旧输出未声明依赖时仍串行，不由后端猜测。trace 使用 `tool_batch_started` 和 `tool_batch_completed` 记录波次与耗时。

12. 记忆与用户画像

12.1 三层记忆

层级	生命周期	例子	写入方式
任务记忆	当前局/当前目标	“这局不和 C 打”“最多打 4 小时”	主 Agent 调 `record_user_memory` 立即写入
会话 checkpoint	跨多轮但属于该会话	当前目标、已确认事实、待问问题	预算触发的摘要模型写入
长期画像	跨会话稳定存在	95% 打 0.5、通常一个人、可烟	候选提炼 + 证据 + 高置信度后晋升

12.2 为什么“中途说不和 C 打”不能等会话结束

如果只在会话结束后提炼画像，当前候选搜索可能在几十秒内就把 C 推荐出来。因此涉及本任务决策的事实要先写任务记忆，立即参与本轮检索；是否属于长期稳定关系，再由后续证据决定。

```
TaskMemory:
  scope_type: game
  scope_id: game_123
  subject_id: user_A
  key: avoid_player
  value: user_C
  evidence_message_id: msg_456
  expires_at: game archive time
```

12.3 长期画像如何写入

1. 模型发现用户明确表达了可能稳定的事实，调用记忆工具保存“候选”。
2. 候选包含原文证据、适用范围、提议值和置信度。
3. 会话结束或达到稳定节点后，记忆提炼器结合多轮证据和现有画像去重、处理冲突。
4. 只有置信度达到高阈值（建议 0.9）且证据明确，才自动晋升；否则保留待人工或更多证据。
5. 用户最新明确表达优先于历史推断；所有变更记录来源 trace，可撤销。

“今天无烟也行”可能只是本局条件，不应覆盖长期“抽烟”画像；“我以后都不打有烟局了”才是长期候选。Scope 是比置信度更先要判断的字段。

12.4 关系记忆

用户关系不是单个客户的标签，而是两人之间的边。关系表使用稳定 pair key，记录共同打过次数、明确回避、来源证据和更新时间。候选搜索会使用关系；客户可见回复只能使用公开昵称，不能泄露老板的私有备注。

13. 领域模型

13.1 GameRequirement 不是一句拼接文本

维度	示例	设计说明
玩法	杭麻、财敲、川麻、幺鸡、红中	财敲是杭麻子玩法，不等于重庆麻将
底注	0.5、1、2、10	与封顶分开存储
封顶	16、32	川麻 `2-16` 拆为 base=2、cap=16
开始模式	固定时间 / 人齐开	人齐开也必须有最晚有效时间
开始与结束	16:00 至 20:00	每个局都有计划区间和过期时间
时长	固定 4 小时 / 可协商 / 通宵	不能把“通宵”原样暴露成内部枚举
烟况	无烟 / 有烟 / 都可 / 可室外抽	偏好和本局硬约束分开
席位结构	当前 2 人、缺 2 人	联系人数量不等于席位数量
候选约束	最好一男一女、回避某人	软偏好与硬冲突分别处理

13.2 173 / 272 / 371

它们是人数短码，不是玩法：

- `173`：当前 1 人，缺 3 人。
- `272`：当前 2 人，缺 2 人。
- `371`：当前 3 人，缺 1 人。

模型负责结合麻将语境理解短码；后端领域对象存标准字段 `known_player_count` 和 `needed_seats`，不会长期依赖字符串“371”。

13.3 川麻 216 / 232

`216` 通常归一为 `2-16`，即底注 2、封顶 16；`232` 归一为 `2-32`。系统将它们拆成 `base_stake` 与 `cap_amount`。如果当地规则对“216”还有“2 的底、4 番满”等等价说法，老板侧只需保存可运营的规范值；只有歧义会影响匹配或结算时才追问。

13.4 Party 和 Participant

一个微信联系人可能代表两个人。Party 表示“由谁与老板沟通、占几个席位”，Participant 表示已知的实际玩家。未知同伴可以用 `anonymous_seat_count` 占位，后续再补身份。

系统业务运营者始终是麻将馆老板；当前领域模型中的 `organizer_id` 表示发起组局需求的客户和首位参与方，不表示运营者。人数计算必须把请求方的席位计入。

14. 状态机与生命周期

14.1 局状态

forming 待补充 → inviting 邀约中 → ready 已成局 → finished 已结束

任何活跃状态都可以在用户取消、过期、房态失败或所有参与方退出后进入 `cancelled`。状态迁移由后端白名单控制，模型只能提出 `update_game_status`。

14.2 邀约和外发状态

对象	状态	关键语义
InviteDraft	pending_approval / sent / confirmed / declined / negotiating / no_reply / superseded	候选人是否被邀请及其响应
OutboundDraft	pending_approval / sent / cancelled / superseded	客户可见文本的发送资格
PendingInputBatch	pending / processing / completed / ignored / failed	碎片输入延迟处理状态

14.3 时间与自动归档

- 固定时间局保存 `planned_start_at`、`planned_end_at` 和 `expires_at`。
- 过去的开始时间不能静默沿用；结合当前时间判断是笔误、明天还是其他时间。
- “人齐开”不能永久活跃，默认最大等待窗口为 4 小时；到期仍未组成则取消归档。
- 用户明确说不打了，立即移除其席位；无人继续参与或目标失效时归档局。
- 搜索当前局只返回仍在有效窗口且有可用席位的活跃局，下午六点不会搜到下午两点已失败的局。

14.4 未来预约与持久化唤醒

用户预约明天或更晚的固定时间时，系统当轮就创建 `Game`，并按 `planned_start_at` 放入今天/明天局列表；不会把“明天再处理”只存在对话记忆中。系统派生 `recruitment_opens_at = planned_start_at - 2h`，在此之前状态为 `scheduled`，允许公开展示和查询，但统一禁止主动私聊候选人。

```
create_game
  → persist Game
  → persist activate_game_recruitment(due_at=T-2h, task_context_id=source)
  → scheduler atomic claim + lease
  → game_recruitment_window_opened
  → re-enter main Agent with current state
  → search candidates / draft invites / stop dynamically
```

任务保存在 SQLite，进程重启后仍可继续；多节点通过原子领取保证只有一个执行者；节点宕机后 lease 过期可重领，失败按上限重试。局已满、取消或结束时，关联任务同步完成或取消。内部唤醒根据来源任务恢复 checkpoint 或受限原文并保留完整 trace，但不会把内部执行摘要发给客户。

持久化保证重启后任务不丢失，但准时执行仍需要至少一个调度器实例常驻运行。本地 Mac 长时间休眠时不会轮询；恢复或重启后，已过期但未完成的任务会立即被原子领取。若要严格准点，应使用常驻服务或外部调度基础设施。

14.5 人数是派生事实

```
occupied_seats =
    sum(party.seat_count for joined_or_confirmed_parties)

remaining_seats = max(0, capacity - occupied_seats)
```

界面、搜索结果和回复都必须使用同一派生逻辑。任何候选确认都在事务中更新 party 状态并重新计算，不能让模型根据“联系人有几个”自行加减。

15. 具体业务场景：问题、设计与解决路径

本章不是罗列 prompt 规则，而是说明每类真实运营问题应由哪一层负责。每个场景都按“输入 → 上下文 → 模型计划 → 工具/状态 → 回复 → 保护与评测”展开。

场景组	核心问题	主要负责层
意图与碎片输入	什么时候等用户补充，什么时候立即行动，如何理解短回复和错别字	Input Aggregator + Gate + 主模型
查局、建局与房态	咨询不是建局，现成局优先，房满和过期如何处理	主模型 + 查局/房态/建局工具
人数、候选人与协商	席位如何计算，候选如何排序，冲突条件如何协商	领域模型 + 搜索工具 + 状态机
记忆、关系与隐私	老客户默认、临时约束、长期画像和公开信息边界	三层记忆 + 关系表 + 审查器
引用、闲聊与打断	业务和闲聊穿插、引用回复、补充消息覆盖旧结果	引用契约 + 会话版本 + 分流模型
异常与恢复	模型超时、非法输出、工具失败、循环和并发	Contract + Gateway + Progress Monitor

15.1 意图与碎片输入

场景 1：弱意图“今天下班有人打麻将吗”不能静默

问题	早期系统把它判断成信息不足并静默，但对老板而言这是明确的潜在客户咨询。
模型上下文	当前时间、发送者画像、近期对话、活跃局摘要和可用的 `search_current_games`。
计划	先把“下班”理解为时间范围，查询近期可用局；有匹配则推荐，无匹配再问是否需要组局。
工具	`search_current_games`。用户还没确认新建时，不调用 `create_game`。
回复	有局：“六点有个 1 块有烟的，371 了，打吗？”；无局：“现在没有诶，要组一个吗？”
设计依据	意图是否可行动与槽位是否齐全是两回事。咨询现有局只需要足够的搜索范围，不需要先补齐新建局全部字段。
回归点	不得回复“我只能处理组局”；不得因为缺玩法、人数就静默；不得自动建局。

场景 2：多轮碎片“老板 / 今天下午 / 有没有打麻将的 / 0.5 或 1 / 烟都可”

问题	逐条进入主流程会重复追问；全部等待又会让明确意图迟迟无响应。
聚合	按 `conversation_id + sender_id` 将原始消息加入同一批次，每条新消息重置 30 秒静默时间和批次版本。
门禁决策	“老板”“今天下午”返回 wait；出现“有没有打麻将的”后可以识别业务意图。若模型已经开始生成但尚未外发，后续两条会使旧版本作废并重跑。
上下文	主 Agent 看到聚合后的 `raw_messages` 和所有 source message ids，不是五次孤立请求。
结果	查询条件为下午时间范围、档位集合 `[0.5, 1]`、烟况 `any`，不把“都可”误写成两个互斥局。
保护	延时批次持久化；旧 run version 无法外发；群聊不同用户不聚合。
回归点	一个业务输入只生成一条最终回复；补充字段必须进入查询；旧草稿状态为 superseded。

场景 3：信息已足够时不能机械等待 30 秒

输入	“下午两点，0.5，无烟杭麻，173，四小时。”
判断	Input Gate 返回 business。时间、玩法、档位、烟况、人数和时长均可执行，不进入 wait。
计划	查房态和现有局，根据结果匹配或建局。
回复	不重复复述全部条件，只需“好，我帮你问问。”或给出现成局。
回归点	明确输入的首字节处理不应平白增加 30 秒；静默是输入完整性策略，不是全局防抖。

场景 4：静默过后仍只有“帮我组个局”

问题	用户没有继续补充，系统不能再次 wait 形成永久无回复。
上下文	门禁收到 `has_already_waited=true`，主模型同时看到画像和当前局池。
处理	老客户可用高置信度画像填默认；无画像或关键差异很大时，用一句自然追问收集最重要的信息。
回复	“今天大概几点？打杭麻还是川麻？”而不是列出“还缺时间、玩法、档位、烟况、人数”暴露槽位系统。
回归点	最多经过一次静默；追问数量受限；不凭低置信画像直接产生副作用。

场景 5：错别字“通宵0。5有人吗”“0，5”

历史问题	正则把少数标点写死，仍会漏掉新的输入形式，且可能误改其他数字。
当前设计	原文、麻将语境、用户常打 0.5/1 的画像交给模型做语义归一，输出结构化 `stake=0.5` 和置信度。
工具	使用结构化 stake 查询，不再把原始“0，5”传给搜索工具。
不确定时	若上下文中也可能表示“0 和 5”，模型应追问，而不是后端强制替换。
回复	“通宵 0.5 的暂时没有，要组吗？”
回归点	测试多种中文标点、空格和语音转写；不新增逐字符业务 if-else。

场景 6：“四点”应结合当前时间消歧

输入	下午 13:59 发送“4 点无烟 0.5，173，5h”。
错误表现	机械追问“上午还是下午”，随后又误问“明天吗”。
正确推理	ContextBuilder 提供当前本地时间和时区。四点在未来约两小时，日常语境默认 16:00；凌晨 4 点显著不合理。
边界	如果当前已经 16:30，文本中的“下午两点”已过去，则必须确认是明天、发错还是改其他时间。
回复	13:59 时无需问上午/下午，直接按 16:00 处理；已过期时问“你说的两点是明天吗？”
回归点	时间解释要使用发送时间，不使用测试机器错误日期；过去时间不落库为活跃局。

场景 7：短回复“六点”“可以”“组”必须结合双方历史

问题	只有用户历史，没有 Agent 上一轮回复，模型不知道“可以”在确认什么。
上下文	`recent_conversation` 同时包含 user 与 assistant turns；引用消息优先于普通近期上下文。
示例	Agent：“现在没有，要组一个吗？” 用户：“组。” → 目标变为明确组局，而不是再次问“要组吗”。
下一步	若画像足够则查房/建局/找候选；若关键条件仍不确定，则只追问真正缺失的决策字段。
回归点	短回复不能被当成独立 unknown；不得回复“先留意下”却没有后续工具动作。

场景 8：闲聊不能触发“好，我帮你问问”

问题	朋友说“要人性化点”“哈哈”，旧系统反复返回业务承诺。
分流	输入门禁结合当前消息、引用和未完成任务判断 casual。闲聊走轻量生成器，不提供建局等副作用工具。
上下文隔离	闲聊 turn 仍进入近期对话，但不会直接更新局状态或长期画像；下一次业务查询仍可读取活跃局。
审查	闲聊文本同样必须经过客户可见内容审查，不能承认自己是 AI 或泄露其他用户。
回归点	闲聊和业务可穿插；闲聊不污染业务槽位；后续“现在几个人了”仍能回到之前的局。

15.2 查局、建局与房态

场景 9：咨询现有局时先查，不要来一个人就建一个局

输入	“现在有没有 0.5 无烟人齐开的？”
模型计划	目标是咨询，不是创建。第一步调用 `search_current_games`，搜索近期可加入局。
有匹配	返回后端筛选后的 actionable matches 和 join projection，模型直接介绍最合适结果。
无匹配	回复“现在没有诶，要组一个吗？”并等待用户明确确认。
状态	在用户说“可以/组一个”之前，不生成 Game，不搜候选，不创建邀约。
回归点	局看板不能因每次咨询产生垃圾局；`inquire_existing_game` 与 `find_players` 要分开。

场景 10：现成局搜索使用时间窗口和画像做邻近推荐

输入	“现在有没有人齐开 0.5 的？”当前 17:00。
搜索范围	人齐开表达“越快越好”，一般查询 17:00 至 18:30 左右的可加入局，而不是只做精确时间相等。
降级匹配	若没有 0.5，但有 18:00 的 1 块有烟 371，且画像显示用户能打 1 块、可烟，工具可以把它作为较低分可行动结果。
回复	“0.5 的暂时没有，六点有个 1 块有烟的，371 了。”
工具契约	`matches` 已是后端按领域策略筛选的可推荐结果；模型不能再用原始精确时间自行推翻。
回归点	不能明明有 18:00 局却回复没有；回复需要说明偏离条件，但不用暴露评分。

场景 11：老客户“帮我约个 6:30 无烟的”可使用画像默认

画像	95% 打 0.5、95% 一个人；1 块时通常会明确说明。
模型判断	把玩法默认当地杭麻、档位默认 0.5、party size 默认 1，并在 reasoning 中标注来源为高置信画像，不向客户复述内部概率。
计划	先查 18:30 附近无烟局；没有才准备组新局。
现成结果	若 19:00 有个 371，直接问“七点三缺一，可以不？”无需重复“0.5 无烟”。
边界	画像默认只用于低风险检索或可逆草稿；高差异或低置信字段继续追问。
回归点	不再机械问“打多大、几个人”；工具查询参数必须记录默认值来源。

场景 12：明确组局短码“五点 0.5，173，无烟”

识别	固定时间 17:00、档位 0.5、当前 1 人、缺 3、人无烟。这里已经明确到足以开始组局。
计划	查房态 → 查可拼局 → 无匹配则创建局 → 搜候选 → 创建邀约草稿。
状态	请求方立即作为 party 占 1 席；新局状态从 forming 进入 inviting。
回复	“好，我帮你问问。”
保护	跨工具一致性保证查询按 17:00 后，创建不能变成人齐开；候选草稿不能写“缺 1 位”。
回归点	对应 badcase：明确组局误分类、查询与创建参数漂移、请求方未计入席位。

场景 13：用户只说“下午两点 0.5 无烟杭麻，帮我组一桌”

问题	不能把“组一桌”误解析成当前 3 人、缺 1 人。
画像策略	老客户有高置信“通常一个人”可默认 party size=1；新客户没有证据时人数未知。
回复	新客自然问“你一个人吗？”；不要在界面伪造缺口。
建局时机	人数是计算缺口和邀约数量的关键字段。未确认且无可靠画像时，可先查现有局，但不应以错误人数创建新局。
回归点	当前人数显示 null/待确认，而不是 3；第二轮“我这边两个人”要覆盖为 party size=2。

场景 14：第二轮补充“我这边两个人”必须更新同一目标

绑定	通过 conversation、sender、近期目标和未完成计划，把短句绑定到当前组局需求。
动作	更新 party 的 `seat_count=2` 或创建时使用 2；重新计算缺口为 2。
上下文	下一 step 的 `active_parties` 和 `sender_active_game_memberships` 立即反映 2 席。
回复	信息齐后继续执行，不再次问“你一个人吗”。
回归点	第二轮槽位覆盖第一轮未知/默认；数据库和界面人数一致。

场景 15：房间已满，用户要求五点开

输入	16:00 用户要求 17:00 开局，但所有房间最早 18:00 空闲。
第一工具	`check_room_availability` 查询计划时段及需要的结束时间。
决策	房态是硬约束，不能先摇人再承诺五点有房。工具返回最早可用区间。
回复	“五点房间满了，最快六点有房，六点可以吗？”
后续	用户接受后再调整目标、查局或建局；拒绝则结束，不留活跃垃圾局。
回归点	不能确认房间、不能创建冲突预订；房态查询和预订分成两个工具。

场景 16：“人齐开、时间再商量”是合法时间模式

语义	用户不是缺时间，而是明确选择 `start_mode=asap_when_full`；人够后再协商具体到店时间。
领域字段	客户可见文案用“人齐开”，内部枚举可以是 `asap_when_full`，但不能直接发给客户。
生命周期	设置最晚有效时间，默认不超过 4 小时；超过仍未组上自动取消归档。
候选邀请	“0.5 无烟，人齐开，打吗？”不出现英文枚举。
回归点	不得追问不存在的固定时间；不得永久留在局池。

场景 17：已经过去的局不能被再次搜索

例子	下午两点找人失败，用户明确不打；晚上六点再次问有没有局。
状态	原局进入 cancelled，记录失败原因：未补齐/用户取消/时间过期。
查询	`search_current_games` 只返回 active 且当前时间落在有效窗口、仍有席位的局。
分析价值	失败原因可用于复盘：时间太近、档位冷门、烟况冲突、候选疲劳等。
回归点	18:00 查询不得返回 14:00 已失败局；归档数据仍可审计但不可行动。

场景 18：川麻“232，272”已明确，不能再次追问人数

结构化结果	玩法川麻、底注 2、封顶 32、当前 2 席、缺 2 席。
历史错误	模型把“一个微信联系人”和“当前两名玩家”混为一谈，后续非法 JSON 修复又改成追问人数。
数据建模	联系人 identity、party seat count、已知成员 identity 分开；272 是 party/局席位事实，不要求存在两个微信联系人。
回复	条件齐全时直接查局或组局，不问“你是一个人还是两个人”。
回归点	Contract 修复不得改变业务字段；工具参数显示 `known_player_count=2`。

15.3 人数、候选人与协商

场景 19：候选人推荐不是只看玩法

候选特征	玩法、档位、常来时间、烟况、响应率、最近邀约、最近到店、疲劳度、性别软偏好、关系冲突。
确定性排序	工具根据结构化字段计算分数和原因，例如玩法 +30、档位 +20、时间 +20、烟况 +10、24 小时未邀约 +10、近期已邀约 -20、明确不玩 -50。
模型职责	提出搜索条件、决定邀请哪些候选人及是否分批；不自己在 prompt 里扫描 50 份画像算分。
解释	老板侧显示“常打杭麻、0.5、下午常来、近期未邀约”；客户侧不暴露评分。
回归点	候选池不能全是与性别偏好冲突的人；明确 avoid 关系必须硬过滤。

场景 20：“最好再来一男一女”是软目标，不是无限阻塞

输入	“四点 0.5 无烟，272，最好再来一男一女。”
建模	人数 2 缺 2 是硬约束；性别组合使用 candidate preference，除非用户明确说“必须”。
搜索	先提高一男一女组合分数；若候选不足，可以返回替代组合并让老板决定是否继续。
隐私	候选性别用于排序，不在外发中解释“因为你是男/女所以找你”。
回归点	不能因为 mock 数据全是女性就假装满足；工具要返回未满足的软约束。

场景 21：发起客户必须计入席位

历史错误	张哥让老板组局后只写到 organizer 字段，未进入 participants，系统显示确认 0 人。
领域解释	老板是实际运营组织者；客户是 requesting party 和确定参与者。请求“帮我组局”即表示本人要打，除非明确代表别人。
落库	`create_game` 在同一事务中创建 game 和 requesting party，默认 seat_count 来自短码/画像/明确人数。
结果	173 创建后 occupied=1、remaining=3；再有一位确认后 remaining=2。
回归点	人数、看板、搜索 join projection、回复全部使用 party seat count，不读 organizer 字符串。

场景 22：一个联系人代表两个人

输入	lin01 回复“我们两个人可以来”。
动作	`record_candidate_reply` 写 confirmed，同时保存 `seat_count=2`、联系人和可选 known_member_ids。
并发事实	更新在事务中重算剩余席位。下一位 k01 查询时，工具拿到最新 committed 状态。
回复投影	工具返回 `remaining_before_join`、`remaining_after_join`、`would_fill_game`，模型不用心算。
回归点	不能把 lin01 算一人；不能把 k01 加入前缺口说成加入后缺口；局满时原子拒绝多余确认。

场景 23：候选人明确回复“可以/打！”

引用优先	若候选回复某条 invite，引用绑定直接给出 game_id 和 invite_id。
语义	“可以”“打”在邀请上下文中为 accepted/confirmed，不是待确认。
动作	调用 `record_candidate_reply`；事务更新 invite、party、game remaining seats。
老板回复	“好，加你 272 了。”短而明确；数字来自更新后的局状态，而不是邀请创建时的旧缺口。
回归点	刘姐确认后 Amy 再确认，不能仍回复同一个 272；每次使用最新工具结果。

场景 24：候选人说“最快四点半”需要发起协商

输入	原局计划 16:00，候选人：“可以倒是可以，但是我最快四点半。”
状态	invite 进入 negotiating，不能直接 confirmed，也不能擅自把局时间改成 16:30。
计划调整	主 Agent 识别需要征求现有参与方，创建给已确认 party 的协商消息草稿。
回复候选人	“我问下他们四点半行不行。”
后续	全部接受才更新局时间；有人拒绝则继续找人或提出替代方案。
回归点	不能只回复“我问问”却没有消息草稿；不能静默修改所有人的时间。

场景 25：候选人要打 6 小时，原局约 4/5 小时

语义	这是条件协商，不是普通确认，也不是闲聊。
任务记忆	记录候选人的本局时长要求，scope=game，不立即写长期“永远只打 6 小时”。
工具计划	检查房间结束时间是否允许，再向已确认参与者生成协商草稿。
回复	“我先问下他们六小时行不行。”
回归点	不能直接说“加你 371 了”；协商未完成前席位状态保持 negotiating。

场景 26：疲劳度和重复邀约

问题	张哥早上打过一局，六小时后又主动让老板组；另一些客户一天通常只打一局。
画像	保存日内频率、最近到店、主动请求与被动邀约差异。用户主动请求不应被疲劳度直接拦截。
排序	补人时优先玩法/档位/时间匹配且近期未被打扰的人；刚拒绝或刚被连续邀约的人降分。
例外	画像显示一天多局或用户主动发起时，疲劳惩罚降低。
回归点	疲劳是排序特征，不是僵硬“24 小时只能打一次”的规则。

场景 27A：同一弹性客户暂时出现在多个互斥候选局

输入	A、B 原本是人齐开；A 最晚 17:30 且只能 4 小时，C 希望 18:00 打 5 小时，B 两边都可以。
错误方案	强行合并成一个折中局，或者因为 B 已出现在第一个待组局就永远不允许他进入其他方案。
领域建模	建立 17:30 的 A+B 方案和 18:00 的 B+C 方案。每个方案是独立 Game；B 在多个 forming/inviting 局中表示临时占位，不表示最终承诺。
提交规则	任一方案先补齐后进入 ready；同一写事务把 B 从所有时间冲突的其他待组方案改为 superseded，并重新计算失败方案缺口。
扩展性	B 可以临时存在于两个、五个或更多候选方案；算法按时间窗口和共享 customer_id 计算，不依赖 A/B/C 的业务分支。
并发	SQLite 在读取跨局不变量前使用 BEGIN IMMEDIATE；两个方案同时抢最后一席时只能有一个 ready。
回归点	胜出局保留 B，失败局释放 B 且继续找人；非重叠时段允许 B 分别成局；私聊和关系约束不跨 conversation 泄露。

场景 27：局满时的最后一席并发确认

竞态	两个候选人几乎同时回复“可以”，局只剩 1 席。
错误方案	先读 remaining=1，再分别写 confirmed，会把局变成 5 人。
当前方案	在 SQLite 写事务中读取并更新 party，使用原子条件和唯一约束；先提交者占位，后提交者得到 game_full。
模型恢复	失败工具结果回喂，主 Agent 对后一位生成礼貌回复：“刚好齐了，下个局我再问你。”
回归点	确定性并发测试同时发起多个确认，最终 occupied 不超过 capacity，只有一个成功。

15.4 记忆、关系与隐私

场景 28：无画像用户 A 说“我不和 C 打”

立即影响	写任务记忆 `avoid_player=C`，当前查局与候选搜索必须排除含 C 的局。
长期候选	同时可建立关系候选，附带原文证据和置信度，但不因一次临时矛盾永久写死。
摘要	若对话变长，checkpoint 的 facts 保留“当前目标回避 C”；局归档后任务记忆过期。
晋升	用户明确说“以后都不和 C 打”或多次表达，才晋升为长期 `avoid_playing=true`。
回归点	压缩后仍不能推荐 C；新会话是否继续回避取决于长期关系是否已晋升。

场景 29：用户问“张哥是谁”

隐私原则	不能说“张哥是组这个局的人”暴露内部角色；不能泄露老板的备注、联系方式或画像。
关系查询	读取询问者与张哥的关系：是否打过、是否明确回避。
回复	没打过：“你们之前没一起打过。”；打过且无冲突：“之前一起打过。”；明确回避：“这个局有张哥，你之前说不和他打，要给你另外组吗？”
边界	只有业务相关且最小必要的信息可以公开，内部 organizer/requesting party 字段不可直接解释给客户。
回归点	客户可见审查器需检查角色泄露和私有备注；话术生成器不能自行补人物关系。

场景 30：“现在几个人”与“都有哪些人”是不同授权

问人数	只回复“现在两个，还差两个”，不主动吐出姓名。
问具体人	可返回参与者对外公开的微信昵称；老板私有备注不能给。
匿名席位	若某联系人代表一位未知同伴，可以说“现在两个”，但不能虚构同伴姓名。
审查依据	`active_game_visible_summaries` 区分 public nickname 和 private remark，审查器只允许前者。
回归点	普通局介绍不列参与者；明确追问才最小披露。

场景 31：本地玩法默认与用户专属玩法

地区先验	杭州通常默认杭麻；四川通常默认川麻。地区先验是弱默认，不覆盖用户明确表达。
画像覆盖	某客户长期只找老板约红中，即使本轮没说玩法，也可优先填红中。
冲突处理	张哥画像同时有杭麻和川麻，但在杭州若要川麻通常会明确说，因此没说时默认杭麻，不追问“财敲还是川麻”。
证据	模型上下文标明每个默认来源和置信度；工具调用保留 requirement 来源审计。
回归点	`cq` 在本项目词典中是杭麻“财敲”，不是重庆麻将。

15.5 引用、闲聊与打断

场景 32：候选人引用邀约回复“可以”

引用解析	渠道层提取 quoted message id，查询 `runtime_message_references` 找到对应 invite/game。
优先级	明确引用高于“最近一个活跃局”。模型收到 `primary_binding=quoted_message`。
动作	只更新被引用的邀请，不影响该用户在其他局的状态。
回复	使用该局最新剩余席位，如“好，加你 272 了。”
回归点	多局并存时不串局；引用无法解析时不能用短回复推进副作用。

场景 33：未解析引用“:人一直 / 一致”不能误推进局

历史问题	WeChat 转发/引用文本解析失败，短句被绑定到附近活跃局，误判为确认。
引用契约	`quoted_message_context.resolved=false` 时，模型不得根据短文本和最近活跃局执行状态写工具。
处理	可作为闲聊回复、要求澄清或忽略；只有新的明确业务证据才能推进。
回归点	对应 badcase `unresolved_quote_bound_to_active_game`，必须验证局参与人数不变。

场景 34：老板已手动回复，Agent 尚未发送

输入	系统监听到老板自己的外发消息，或控制台标记已人工处理。
动作	对应会话版本推进；与当前目标冲突的 pending outbox 标记 superseded/cancelled。
幂等	若该人工消息与草稿文本相同，也不能再由 Agent 自动发一次。
审计	保留“由人工完成”的 sender/source，不把人工文本冒充模型输出。
回归点	外发 worker 在发送前再次检查 run version 和 draft status，而不是只在生成时检查。

场景 35：闲聊穿插后继续问局状态

对话	组局进行中 → 聊 AI 是否能替代人工 → 问“所以现在有人了吗”。
隔离	闲聊不清空 active game、task memory 或 checkpoint；也不把“AI”话题写入组局需求。
恢复	最后一句是明确业务状态查询，主 Agent 使用 sender membership 和活跃局工具返回最新人数。
回复	“还没有，还差俩。”或“现在两个了，还差两个。”
回归点	golden dataset 同时验证闲聊质量和后续业务状态不丢失。

场景 36：用户补充消息到达时停止旧主流程

竞态	主流程已经查到一个有烟局并准备回复，用户补充“必须无烟”。
控制	新消息提升 conversation version；旧 run 在写草稿、审查或外发前都检查是否仍为 current。
结果	旧回复即使模型已经生成，也不会外发；新运行按无烟条件重新搜索。
分布式	版本保存在共享事实源，节点 A 的旧运行和节点 B 的新输入仍能比较同一版本。
回归点	并发测试人为延迟旧 run，确保只发新版结果。

15.6 模型、工具与执行异常

场景 37：模型输出非法 JSON 或超长自我纠结

检测	LLM Client 记录原始响应、耗时和解析错误；Contract Parser 在工具执行前拒绝。
恢复	一次结构修复或把具体契约错误回喂模型；修复只改结构，不能推断业务字段。
超时	主调用默认端到端 45 秒；排队等待也计入，不允许排队后再获得完整 45 秒。
终止	剩余预算不足或连续失败时转人工/保守回复，并记录 trace 和 badcase。
回归点	“232/272”不能在 JSON 修复后被改成询问人数；副作用工具一次都不能执行。

场景 38：工具调用失败或参数非法

ToolResult	返回 `allowed/called/error/deduplicated/result/state_transitions`，而不是抛异常结束整个请求。
模型响应	根据错误修参数、先调用前置工具、追问用户或更换方案。
例子	未查房态直接 reserve → 工具拒绝并提示 prerequisite；模型下一步先 check_room_availability。
重试边界	只重试幂等读操作或明确可恢复错误；状态写依赖幂等账本，不能盲目重放。
回归点	错误必须出现在 trace 和下一轮上下文；不能返回空白或直接转人工。

场景 39：模型重复查同一个局，无进展消耗 token

指纹	对动作、工具名、稳定参数和去除时间戳等波动字段后的观察结果生成 fingerprint。
检测阈值	相同 observation 重复 2 次、连续无进展 2 次、或长度不超过 3 的短周期重复。
第一次	向模型注入进展错误，要求解释为什么重复并重规划；最多允许 1 次 progress replan。
仍失败	停止 loop，返回安全终态或 needs_human；不继续到 `max_steps=8` 才发现。
回归点	badcase `agent_repeats_tool_without_progress`；评测断言工具调用次数和 stop reason。

场景 40：内容审查误拒绝或话术改变事实

语义漂移	话术模型把“已创建草稿”写成“已经发给 8 个人”，或把“还有 2 席”写成“还差 1 人”。
审查输入	原始业务回复、话术文本、权威工具结果、可公开局摘要和接收者身份。
审查输出	逐 item 判断 privacy、internal detail、unsupported action、semantic fidelity，并给安全改写。
回喂	拒绝结果作为 loop 内工具结果回给主模型，不是简单替换成固定“转人工”。
评测	自然确认“没问题”不能被词表误判；并发采样下回复事实要与工具结果一致。

16. 并发、一致性与幂等

16.1 同一会话串行，不同会话并行

同一 `conversation_id` 的输入必须按版本顺序处理，否则“我两个人”“改成无烟”可能被旧消息覆盖。不同会话没有业务依赖，可以并行调用模型和工具。

层级	机制	解决的问题
消息级	`runtime_message_results` 以 source message id 去重	渠道重投同一消息不重复调用模型
会话级	本地文件锁或 Redis 分布式锁	同一会话不乱序，不同会话可并行
运行级	conversation version + run id	新输入使旧未发输出失效
工具级	后端生成的 idempotency key + ledger	重试不重复建局、邀约或记录回复
数据级	SQLite 事务、唯一约束、条件更新	最后一席、房间和开放邀约不超卖
跨局级	共享参与者时间窗竞争 + 原子释放	多个候选局只能有一个冲突方案最终承诺同一客户

16.2 为什么两次相同文本不会自动有同一个 source_message_id

source id 来自渠道事件，不来自文本。用户真实发送两次“可以”应有两个不同事件 id；网络重试同一事件才会保留同一个 id。文本相同不等于消息重复，所以不能用纯文本哈希做强幂等。

语义去重只作为软判断：在短时间、同会话、同 sender、同业务引用、同文本和相同状态下，可以提示模型这是重复表达；但真正副作用仍靠工具幂等和状态唯一约束兜底。

16.3 分布式节点挂掉如何恢复

1. 原始消息和 pending input batch 已持久化，不只在进程内存。
2. processing lease 到期后，其他 worker 可以重新领取延时批次。
3. 工具幂等账本区分“尚未执行”和“已成功但响应丢失”。
4. 数据库是权威状态，重启后的模型上下文重新读取最新 game/party/invite。
5. outbox worker 发送前检查草稿状态、当前会话版本和通道开关。

> 当前单机默认使用 SQLite 与文件锁。真正多机部署时必须把协调锁、数据库和 outbox 迁到所有节点共享的基础

> 设施；仅仅复制同一套代码到两台机器并不能获得一致性。

16.4 模型并发背压

主供应商调用由 `BoundedSemaphore` 限制，默认最大并发 3；fallback 默认最大并发 2。排队时间计入端到端 timeout，避免 20 个请求都排队 40 秒后再各自获得新的 45 秒。

```
deadline = now + request_timeout
remaining = deadline - now
acquire_provider_slot(timeout=remaining)
remaining = deadline - now
call_provider(timeout=remaining)
```

这解决了“模型本身没失败，但主模型、话术和审查同时突发，把供应商压到超时，最终全部转人工”的问题。

17. 话术生成与客户可见内容审查

17.1 为什么主模型之后还要话术模型

主模型要同时规划工具、维护状态和满足 Contract，容易产出字段化或客服化文本。话术生成器只负责把已确定的事实压缩成麻将馆老板日常说法，不允许新增动作或事实。

不理想	理想	原因
“当前存在一个 1 元有烟、人齐开、4 小时的局，是否加入？”	“有个 1 块有烟人齐开的，打吗？”	去除客服腔和字段枚举
“好的张哥，我已按 0.5、无烟、两点帮您问人。”	“好，我帮你问问。”	不复述用户刚说过的条件
“已生成邀约草稿给 8 名候选人，等待老板审批。”	“好，有消息跟你说。”	不泄露内部执行和审批
“您是否方便来？”	“两点 0.5 无烟，打吗？”	邀约短而自然

17.2 话术输入必须包含权威事实

- 主 Agent 原始 `reply_to_user`。
- 本轮成功工具结果及状态转换。
- 对接收者允许公开的局摘要。
- 是否熟悉、是否允许称呼；默认不带姓名。
- 老板话术风格：短句、少复述、非客服、不解释系统。

话术生成器不能看到私有备注、无关客户完整画像或内部评分，从输入源头减少泄露面。

17.3 凡是客户可见文本都必须审查

审查覆盖主回复、候选邀约、协商消息、闲聊和自动外发测试，不只审查 `final_reply`。审查契约主要检查：

- 是否泄露 AI、模型、prompt、工具、trace、数据库、token、草稿、审批和候选数量。
- 是否泄露其他客户的私有备注、联系方式、画像或未获授权的姓名。
- 是否声称执行了工具未完成的动作，例如“已经发出”“已经订房”。
- 是否改变人数、时间、烟况、玩法、时长等业务事实。
- 是否出现不必要的客服化表达或过度承诺。

```
{
  "approved": false,
  "needs_human": false,
  "violations": ["unsupported_action", "internal_detail"],
  "item_reviews": [{
    "item_id": "reply_to_user",
    "suggested_safe_text": "好，我帮你问问。",
    "reasoning_summary": "原文声称已发送给具体人数，但工具只创建了待审批草稿。"
  }]
}
```

17.4 审查是 loop 内部工具结果

审查不通过后，结果回喂主模型，让主模型基于违规项和权威事实重写；只有连续无法生成安全文本或涉及高风险时才转人工。硬字符串过滤只做上线前最后兜底，不承担业务语义。

18. 微信通道与外发控制

18.1 WeChaty 在系统中的定位

WeChaty 只是消息通道，不负责业务决策。它监听私聊/群聊事件，把原始消息转发给 `/api/channels/wechaty/raw`，再把经过主流程和审查的可发送结果送回具体 contact 或 room。

```
WeChat event
  → raw JSONL audit
  → normalize envelope
  → self/room/whitelist filter
  ├─ private → input aggregation & gate → main Agent
  └─ managed room → L1 protocol → L2 route → optional main Agent/private switch
  → reviewed outbound
  → WeChaty precise talker/room target
```

18.2 精准回复给谁

- 私聊使用 talker contact id 作为 `sender_id` 和目标地址。

- room id 标识公开房间和看板；进入主 Agent 时，每位群成员使用 `group:<room_id>:customer:<customer_id>` 独立会话，不能共享整段群聊上下文。
- 引用回复保存 source message id 到业务对象的 reference，避免只靠昵称定位。
- 公开昵称用于客户可见文本，稳定 channel id 用于路由；昵称或备注都不是主键。

18.3 群聊三层处理和看板

显式托管群先经过独立群聊域：L1 只处理严格报局和编号认领协议；L2 选择忽略、群内短答或切私聊；只有需要自然语言决策时才进入主 Agent。复杂请求且双方是好友时，后端写入短期 `ChannelSwitch`，只把该客户原文、画像摘要、解析需求和缺失字段送入私聊，不复制房间其他人的消息。

看板不是聊天摘要，而是权威 Game 状态的实时投影。普通变更在 30 秒内合并，座位变化立即排队刷新；每次成功发布保存不可变 `BoardSnapshot` 和编号到 `game_id` 的映射。有引用时按被引用版本精确解析，无引用时只接受最新版，禁止猜测旧编号。

认领在 SQLite 同一写事务中更新参与者并写 `GameClaim` 幂等记录。好友优先私聊结果，非好友只在群里发送最小确认；公开投影不包含参与者身份、微信备注、关系冲突原文或内部席位结构。

18.4 灰度策略

阶段	输入	输出
本机自测	只允许 self message	关闭外发或只发给自己的测试账号
好友白名单	陈子贤、刘峻甫、Ech0、刘臻等映射到稳定 channel id	审查通过后可自动外发
老板试运行	有限私聊/群聊	低风险自动发，高风险待审批
扩大灰度	按群、好友、消息类型放量	基于 eval 和线上指标逐步放开

18.5 外发开关

页面提供通道开关。关闭时仍可采集、推理、生成和审查，但 outbox 不会调用 WeChaty。重新开启前应检查待发草稿是否仍在有效版本，避免把几小时前的回复补发给客户。

18.6 图片和语音

渠道层应先完成多模态预处理：语音下载后经 ASR 得到文本和置信度；图片/表情包经视觉模型得到描述或业务字段。主 Agent 继续接收统一消息信封，并保留原媒体引用。低置信转写不能直接触发高风险副作用。

19. 日志、审计与可回溯

19.1 Trace 贯穿哪些步骤

trace 在消息进入服务器时生成，不由模型提供，也不允许模型修改。单条 trace 至少可看到：

```
input_received
input_fragment_persisted / input_gate_decision
conversation_lock_acquired / run_version_created
context_packed / context_built / context_summary_checked
llm_prompt / llm_response / action_proposed
action_contract_validated / action_contract_repaired
tool_call_requested / tool_result / state_transition
progress_observed / progress_replan
copywriter_prompt / copywriter_result
customer_visible_content_review_result
final_output / outbound_status
post_turn_summary_result
```

19.2 日志格式

```
traceId-yyyy-mm-dd hh:mm:ss-LEVEL: {"step":"...", "content":"..."}
```

结构化 content 包含耗时、模型、预算决策、工具参数摘要、错误类型和实体 id。API key、完整私有画像和敏感凭证必须脱敏。原始 WeChaty 事件单独写 JSONL，便于通道问题和业务问题分离排查。

19.3 状态审计

每次非幂等命中的状态变化写入：

```
{
  "entity_type": "game",
  "entity_id": "game_123",
  "from_state": "inviting",
  "to_state": "ready",
  "reason": "four seats confirmed",
  "trace_id": "trace_xxx",
  "occurred_at": "..."
}
```

审计可以回答“谁让局变成 ready”“哪个候选确认占了最后一席”“哪条补充消息作废了旧回复”，而不只是输出一堆文本日志。

19.4 延迟与成本指标

- 入口到门禁、门禁到主模型、每次工具、话术、审查和总耗时。
- P50/P95/P99 模型调用与端到端延迟。
- 每轮模型调用次数、estimated prompt tokens、实际 usage 和费用。
- 供应商 active/waiting/max concurrency、超时和 fallback 比例。

- 自动回复率、转人工率、审查拒绝率、无进展终止率。

20. Badcase、Golden Dataset 与 Eval

20.1 为什么不能只“测试几句”

模型输出有采样波动，一次正确不代表稳定。每个线上问题先结构化记录，再选择正确修复层：prompt、context、contract、工具语义、数据模型、并发机制或评测器。修复后必须加入回归，避免下次改话术时把人数逻辑改坏。

20.2 三类资产

资产	用途	目录
badcase	保存真实失败输入、实际/期望、trace、根因、修复和状态	`eval/badcases/badcases.jsonl`
golden dataset	保存老板真实对话和稳定期望行为，不要求逐字完全一致	`eval/golden/`
regression	自动验证工具调用、状态、回复约束和禁用动作	`eval/regression/`
few-shot	给模型展示高质量领域理解和话术范例	`eval/few_shot_examples.jsonl`

20.3 当前 22 个已覆盖 badcase

问题族	已归档问题	通用修复层
话术	时长回复系统化、现成局字段直拼、客服化“要加入吗”、话术越权新增事实	话术契约 + 权威结果 + 内容审查
隐私	询问人物时泄露组局角色、未解析引用误绑定活跃局	可见摘要 + 引用契约 + 审查
人数	发起客户未计入、两人 party 只计一席、加入前后缺口混淆、232/272 重问人数	Party/Seat 数据模型 + join projection
参数一致性	四点查询后建成人齐开、模型推翻后端可行动搜索结果	跨工具一致性 + 搜索结果语义契约
输入	明确短码被当咨询、碎片消息处理过早、补充消息未作废旧输出	输入门禁 + 聚合批次 + 会话版本
循环/合同	重复工具无进展、终态字段一致但 status 写错	Progress Monitor + 窄 Contract repair
并发	重复开放邀约、锁命名空间误串行、供应商突发超时、并发回复语义漂移	事务唯一约束 + 锁 namespace + 背压 + 审查
评测器	“没问题”自然确认被严格词表误报	语义断言 + 允许等价表达

20.4 当前验证结果

248 + 1
自动化测试

248 passed，1 skipped。

138 / 138
Agent 确定性回归

9 个核心业务场景的结构化检查。

24 / 0
Badcase 状态

fixed=24，open=0。

7 / 7

确定性并发场景

每场 40 操作：消息去重、会话隔离、最后一席、房态、重复建局、重复邀约和版本原子性。

20 / 20

真实 DeepSeek 并发

74 次模型调用，失败 0；模型 P95 7.58s，场景端到端 P95 24.44s。

20.5 Golden 不等于逐字匹配

真实老板可能回复“ok”“okk”“好的”“可以”。评测应检查业务动作、事实、隐私、长度和风格，不因标点或同义词判失败。对工具和状态使用强断言，对自然语言使用语义/禁用词/最大长度等组合断言。

21. SQLite 与 Redis

21.1 SQLite 是当前事实源

表	用途
`runtime_customers`	长期客户画像
`runtime_customer_relationships`	两人共同打过、回避关系和证据
`runtime_games`	局需求、party/participant、时间和状态 payload
`runtime_invite_drafts`	候选邀约和响应状态
`runtime_outbound_message_drafts`	多通道待发消息与版本
`runtime_rooms` / `runtime_room_reservations`	房间库存和时间段预订
`runtime_conversation_turns`	user/assistant/tool 多轮记录
`runtime_conversation_checkpoints`	会话摘要 checkpoint
`runtime_task_context_checkpoints`	按 task context 归档的历史任务 checkpoint
`runtime_conversation_versions`	新输入打断旧输出的版本号
`runtime_pending_input_batches`	碎片聚合、静默截止时间和处理租约
`runtime_channel_identities`	渠道账号、稳定客户和私聊能力映射
`runtime_group_room_policies`	托管群、看板/外发开关和合并窗口
`runtime_game_conversation_links`	Game 与房间/私聊任务的业务关联
`runtime_group_board_snapshots` / `runtime_group_board_items`	已发送看板版本及编号到 game_id 的不可变映射
`runtime_group_game_claims`	群内认领和来源消息幂等记录
`runtime_channel_switches`	短期群转私聊续接指针
`runtime_message_results`	消息级幂等结果
`runtime_message_references`	渠道消息到 invite/game 等业务对象的引用

`runtime_idempotency_ledger`	工具调用幂等账本
`runtime_task_memories`	任务范围事实
`runtime_pending_memory_candidates`	待晋升长期画像
`runtime_state_transitions`	状态机审计
`runtime_badcases`	运行时 badcase 归档

快速演进的完整对象使用 JSON payload 保存，同时把 `conversation_id`、`status`、`customer_id`、`updated_at` 等查询字段冗余成普通列和索引。这样既避免频繁迁移，又不让所有查询都退化成 JSON 全表扫描。

21.2 Redis 的定位

Redis 不是客户画像的唯一存储。启用时主要负责短期缓存和分布式协调：

数据	结构	说明
会话分布式锁	String + TTL	多节点保证同 conversation 串行
热点画像和活跃局缓存	Hash / String	减少 SQLite 读取，DB 仍为权威
短期聚合消息	List / Stream	可扩展为分布式输入队列
过期扫描索引	Sorted Set	按 expires_at 找到待归档局
短 TTL 去重	String	减少热点重复请求，最终仍靠 DB ledger

Redis 未启用时系统可以单机运行；提示“Redis 未启用”表示使用本地协调与 SQLite，不代表客户画像丢失。页面常客数据来自 SQLite seed/持久化，可能在 Redis 启用时被缓存加载得更快。

22. 预算、超时与关键配置

配置	默认值	目的
`MAHJONG_AGENT_MAX_STEPS`	8	单轮 Agent 最大 loop 步数
`MAHJONG_AGENT_LLM_TIMEOUT_SECONDS`	45s	主调用端到端超时
`MAHJONG_AGENT_MAX_TOKENS_PER_CALL`	24000	主上下文估算预算
`MAHJONG_AGENT_MAX_CALLS_PER_TURN`	8	单条用户消息的模型调用预算
`MAHJONG_LLM_MAX_COMPLETION_TOKENS`	1600	主模型最大生成长度
`MAHJONG_LLM_MAX_CONCURRENCY`	3	主供应商并发槽
`MAHJONG_AGENT_MAX_PARALLEL_READ_TOOLS`	4	单个 Agent 步内并行只读工具上限
`MAHJONG_LLM_FALLBACK_MAX_CONCURRENCY`	2	fallback 并发槽
`MAHJONG_INPUT_QUIET_PERIOD_SECONDS`	30s	碎片输入静默窗口
`MAHJONG_INPUT_QUIET_BOUNDARY_GRACE_SECONDS`	5s	避免静默截止与下一条边界消息抢跑
`MAHJONG_API_BACKGROUND_PROCESSING_ENABLED`	true	HTTP 控制台接收批次后后台执行主 Agent
`MAHJONG_WECHATY_INPUT_GATE_TIMEOUT_SECONDS`	12s	输入门禁超时
`MAHJONG_WECHATY_CASUAL_CHAT_TIMEOUT_SECONDS`	12s	闲聊生成超时
`MAHJONG_CONTEXT_SUMMARY_TOKEN_THRESHOLD`	3000	常规摘要触发阈值
`MAHJONG_CONTEXT_SUMMARY_MAX_INPUT_TOKENS`	6000	摘要模型输入预算
`MAHJONG_CONTEXT_SUMMARY_LLM_MAX_COMPLETION_TOKENS`	1200	摘要输出预算
`MAHJONG_AGENT_REPEATED_OBSERVATION_LIMIT`	2	相同观察重复阈值
`MAHJONG_AGENT_NO_PROGRESS_LIMIT`	2	连续无进展阈值
`MAHJONG_AGENT_MAX_PROGRESS_REPLANS`	1	允许模型自我纠正次数

预算不只限制 token，也限制调用次数、并发和墙钟时间。一个请求即使每次调用都不大，如果主模型、话术、审查重复多次，也可能超出成本和延迟目标。

23. 启动、排障与代码索引

23.1 启动与检查

```
cd "<repository-root>"

# 启动 Web / Runtime
PYTHONPATH=src python scripts/run_agent_app.py

# 页面
open http://127.0.0.1:8790/

# 实时 trace
tail -f logs/agent_runtime_trace.log

# 全量确定性测试
python -m pytest -q

# 评测
PYTHONPATH=src python scripts/run_evals.py

# 并发评测
PYTHONPATH=src python scripts/run_concurrency_eval.py --mode deterministic
```

23.2 排查一条失败回复

1. 用 trace id 搜索完整日志，不只看 final reply。
2. 检查输入门禁是否进入了正确分支，碎片是否聚合完整。
3. 检查 ContextBuilder 是否包含 Agent 上一轮回复、画像、引用和活跃局。
4. 检查模型 Action 是理解错、计划错，还是 Contract 解析失败。
5. 检查 ToolGateway 实际执行的参数和结果，不能只看模型 reasoning。
6. 检查数据状态是否已提交，join projection 是否正确。
7. 检查话术生成器或审查器是否改变了主流程正确事实。
8. 把问题归到正确修复层并记录 badcase，再跑全量回归。

23.3 关键代码索引

主题	路径
启动	`scripts/run_agent_app.py`
HTTP / WeChaty / 输入门禁	`scripts/agent_runtime_app.py`
Runtime 边界	`src/mahjong_agent_runtime/runtime.py`
主循环	`src/mahjong_agent_runtime/loop.py`
动作处理	`src/mahjong_agent_runtime/processing.py`
上下文	`src/mahjong_agent_runtime/context.py`
摘要	`src/mahjong_agent_runtime/summary.py`
工具	`src/mahjong_agent_runtime/tools.py`
工具参数一致性	`src/mahjong_agent_runtime/tool_consistency.py`
无进展检测	`src/mahjong_agent_runtime/progress.py`
话术与审查	`src/mahjong_agent_runtime/copywriting.py`、`visibility.py`
模型客户端	`src/mahjong_agent_runtime/llm.py`
SQLite	`src/mahjong_agent_runtime/sqlite_store.py`
领域对象	`src/mahjong_agent_runtime/models.py`
主提示词	`src/mahjong_agent_runtime/prompts/agent_runtime_system.md`
输入门禁提示词	`src/mahjong_agent_runtime/prompts/wechaty_input_gate.md`
评测说明	`eval/README.md`

24. 当前边界与演进路线

24.1 已具备

- 目标驱动多步工具 loop，工具结果回喂并动态调整计划。
- 碎片输入聚合、闲聊分流、引用消息保护和新消息打断。
- 局、席位、邀约、房间、状态机、任务记忆和长期画像候选。

- 消息/工具幂等、会话顺序、原子抢位、重复邀约和并发背压。
- 上下文预算、预压缩、checkpoint、无进展与死循环检测。
- 客户可见文本审查、外发开关、白名单微信灰度。
- trace、状态审计、badcase、golden、确定性和真实模型并发评测。

24.2 上线前仍要补强

优先级	事项	完成标准
P0	真实老板连续试运行	至少数周、真实流量、统计自动回复率/纠正率/漏回率
P0	外发可靠性	outbox 重试、送达确认、人工消息冲突、断线恢复
P0	数据备份与恢复演练	SQLite 定时备份、加密、恢复时间和数据丢失目标
P0	账号与合规风险	个人微信通道风险评估、白名单、频控、隐私告知
P1	自动画像晋升治理	冲突合并、过期、人工修正、用户删除和证据回溯
P1	持续异步补人策略	在已实现的预约唤醒基础上，支持分批邀约、等待回复与再次唤醒
P1	房态真实接入	门店房间、到店、延时和换房与真实运营同步
P2	语音/图片	ASR/视觉抽取、置信度和媒体审计链路
P2	多租户和多门店	租户隔离、配额、独立画像和规则配置

24.3 已实现的未来预约唤醒与下一阶段

当前已经把未来预约的“立即记局”和“T-2 小时后开始找人”拆成两个可恢复阶段：

```
同步主流程（低延迟）
  查现有局 → 无匹配 → 确认需求 → 回复“好，我帮你看看”
                                  |
                                  ▼
持久化预约任务（已实现）
  建局 → 等到 T-2h → 原子领取 → 唤醒主 Agent → 查候选 → 生成邀约草稿
```

下一阶段是在同一 durable task 基础上继续增加“分批邀约 → 等回复 → 再次唤醒 → 协商 → 成局/超时 → 主动通知”，而不是重新做一套旁路业务流程。

25. 面试讲解模板

25.1 一分钟版本

我做了一个面向麻将馆私域运营的目标驱动 Agent。它不是固定 Workflow：模型会根据用户消息、近期对话、客户画像、活跃局和工具结果，自主规划查局、查房态、建局、找候选人和推进状态。后端不理解每一句业务话术，只负责工具 schema、权限、幂等、事务、状态机和审计，模型不能直接改数据库。系统接入 WeChaty 做了白名单试运行，并建立了上下文摘要、内容审查、无进展检测、trace、22 个 badcase、golden dataset 和并发评测。

25.2 如何解释 ReAct

它属于 ReAct 风格：模型根据当前观察提出动作，工具执行后把结果回喂，模型再观察并决定下一步。但不是“纯 ReAct 文本思维链”，而是使用结构化 Action Contract、目标计划、状态机和受控 ToolGateway 的生产化实现。

25.3 如何解释上下文工程

每个 step 都由 ContextBuilder 动态组装当前消息、近期双方对话、checkpoint、画像、关系、任务记忆、活跃局、席位、待发草稿、工具和上一轮工具结果。近期原文从近到远按约 4000 token 装载；会话达到阈值或完整上下文达到 24000 预算的 85% 时生成摘要 checkpoint，再重建上下文。业务事实永远以数据库和工具结果为准，摘要只负责压缩历史。

25.4 如何解释记忆

我把记忆拆成任务记忆、会话 checkpoint 和长期画像。比如用户中途说“这局不和 C 打”，必须马上写任务记忆，立刻影响候选搜索；会话结束后模型再根据证据提炼长期关系候选，只有高置信和明确长期范围才晋升，避免一次临时偏好污染画像。

25.5 如何解释高并发

同 conversation 用共享锁和版本串行，不同 conversation 并行。消息通过渠道 id 去重，工具通过后端幂等账本去重，最后一席、房态和重复邀请用数据库事务与唯一约束保证。模型端用并发信号量做背压，排队时间纳入端到端超时。当前确定性并发 8/8，真实 DeepSeek 并发 22/22，85 次调用无失败。

25.6 这个项目最重要的设计经验

1. 不要把模型不稳定简单归因为“再加一条规则”，先判断是上下文、数据模型、工具语义还是契约问题。
2. LLM 负责不确定语义，后端负责不可违反的不变量。
3. 所有副作用必须工具化、幂等化、事务化、可审计。
4. Agent 的回复质量依赖权威事实和用户关系，不只是 prompt 写得像人。
5. 没有 badcase 和 eval，就无法证明一次改动是真优化还是换了一种失败。

> **文档事实版本：**2026-08-11。内容对应当前 `mahjong_agent_runtime` 、README、运行时配置、22 个 badcase 和并发评测结果。代码继续演进时，应同步更新本文中的工具、表、预算、状态机和验证指标。